DATASTOCK

Práctica Integradora

ENTREGA 3 – UT3

Acceso a datos

2.º DAM · Curso 2026–2027

IES ISIDRA DE GUZMÁN · ALCALÁ DE HENARES

Contenido

1. Objetivo y alcance 2
2. Qué tienes que entregar (resumen) 2
3. Resultado de aprendizaje y criterios trabajados 2
4. Componentes mínimos que debes desarrollar 3
4.1. Componente de ficheros: lector de productos JSON 3
4.2. Componente de base de datos: consulta de productos 3
5. Integración en DataStock 3
6. Guía de trabajo paso a paso 4
7. Pruebas mínimas obligatorias 4
8. Breve reflexión técnica en README 4
9. Entregables y evaluación 5
10. Lista final de comprobación 5

1. Objetivo y alcance

Esta es la tercera y última entrega del mismo proyecto DataStock. Tras trabajar con ficheros en la UT1 y con MySQL/JDBC en la UT2, vas a mejorar una pequeña parte del código para convertirla en componentes reutilizables. No debes rehacer la aplicación ni añadir nuevas tablas, pantallas o funcionalidades de negocio.

El trabajo es individual. Conserva tu repositorio, tu proyecto Maven con Java 21, el script SQL y los menús existentes.

2. Qué tienes que entregar (resumen)

- Un componente de lectura de productos desde JSON, extraído de tu implementación de la UT1.
- Un componente de consulta de productos en MySQL mediante JDBC, extraído de tu implementación de la UT2.
- La integración de ambos componentes en dos opciones que ya existen en DataStock; no se añaden menús nuevos.
- Tres pruebas documentadas y un README breve que explique las decisiones de diseño.

Importante: solo se refactorizan las dos operaciones indicadas. No es necesario convertir todo el proyecto a arquitectura por capas, crear un framework, ni rehacer el CRUD, las transacciones o los procedimientos almacenados.

3. Resultado de aprendizaje y criterios trabajados

RA6. Desarrolla programas que gestionan información almacenada en bases de datos y ficheros mediante componentes reutilizables.

CE	Evidencia concreta en esta práctica
a) Ventajas e inconvenientes de componentes	Breve comparación razonada en README.
b) Herramientas de desarrollo de componentes	Identificación de IDE, Maven, JUnit y Javadoc y su utilidad.
c) Componentes que gestionan ficheros	Componente JSON que devuelve objetos Producto.
d) Componentes que gestionan BD mediante conectores	Componente JDBC de consulta de productos.
h) Componentes probados y documentados	Tres pruebas y documentación de API.
i) Integración en aplicaciones	Dos opciones existentes del menú usan los componentes.

Esta entrega proporciona evidencias de los CE a, b, c, d, h e i de RA6. Se mantiene el sistema de evaluación establecido en la programación didáctica: práctica 60 % y prueba individual teórico-práctica 40 %.

4. Componentes mínimos que debes desarrollar

4.1. Componente de ficheros: lector de productos JSON

Partiendo del código que ya utilizas para importar JSON en la UT1, extrae la lectura del fichero a una clase reutilizable (nombre orientativo: LectorProductosJson).

- Entrada: ruta del fichero JSON (Path o equivalente).
- Salida: colección de objetos Producto válidos; el componente NO debe imprimir menús ni leer opciones con Scanner.
- Debe informar de errores de fichero inexistente, JSON incorrecto y datos inválidos de forma controlada.
- No debe insertar datos directamente en MySQL ni modificar el inventario por sí mismo.
- La opción 2 del menú principal seguirá importando JSON, pero ahora llamará al componente y añadirá al inventario los productos recibidos.

4.2. Componente de base de datos: consulta de productos

Partiendo de la consulta por código que ya funciona en la UT2, extrae el acceso JDBC a una clase reutilizable (nombre orientativo: ConsultaProductoJdbc).

- Entrada: código de producto.
- Salida: un Producto o un resultado que permita indicar claramente que no existe.
- Utilizar PreparedStatement y ResultSet; obtener la conexión desde la configuración JDBC que ya tienes.
- Cerrar los recursos correctamente; no mostrar mensajes de consola dentro del componente.
- La opción 7 > 5 > 2 (buscar por código) seguirá funcionando, pero llamará al nuevo componente y presentará el resultado.

Puedes usar las clases de modelo ya existentes. No es obligatorio crear interfaces ni nuevas capas adicionales. Si ya utilizas DAO o servicios, adapta estas indicaciones a tu organización sin duplicar clases.

5. Integración en DataStock

La aplicación conservará los menús y funcionalidades de las entregas anteriores. El cambio debe ser interno: el usuario utilizará las mismas opciones, pero la lectura JSON y la búsqueda JDBC estarán delegadas en sus componentes.

```
MENÚ PRINCIPAL
2. Importar productos desde JSON   -> LectorProductosJson
7. Gestionar base de datos MySQL
   5. Consultar y buscar productos
      2. Buscar producto por código -> ConsultaProductoJdbc
```

El menú será responsable de solicitar los datos y mostrar los resultados: [OK] en verde, [ERROR] en rojo y avisos cuando corresponda. Los componentes no deben depender de la consola, para poder reutilizarlos desde otra aplicación o desde una prueba.

6. Guía de trabajo paso a paso

1. Crea una rama o realiza un commit antes de refactorizar. Comprueba que DataStock funciona con la versión UT2.
2. Localiza el código que lee el JSON en la UT1. Trasládalo a LectorProductosJson sin cambiar las reglas de validación.
3. Modifica únicamente la opción de importar JSON para que invoque al nuevo componente. Comprueba que el resultado es el mismo que antes.
4. Localiza la consulta JDBC de búsqueda por código. Trasládala a ConsultaProductoJdbc, separando la consulta de la presentación por consola.
5. Modifica únicamente la opción de búsqueda por código para que invoque al componente. Comprueba los resultados con un producto existente y otro inexistente.
6. Añade documentación Javadoc a las dos clases y a sus métodos públicos: finalidad, parámetros, resultado y posibles errores.
7. Ejecuta las pruebas del apartado 7, registra los resultados y completa el README.

7. Pruebas mínimas obligatorias

Caso	Acción	Resultado esperado
CP-COMP-01	Importar un JSON válido desde el menú principal.	Se cargan en memoria los productos válidos y se muestra [OK].
CP-COMP-02	Importar un JSON inexistente o mal formado.	Error controlado en rojo; DataStock sigue funcionando.
CP-COMP-03	Buscar en MySQL un código existente y otro inexistente.	Se muestra la ficha correcta o un aviso; no hay excepciones sin controlar.

Registra para cada caso: identificador, datos utilizados, resultado esperado, resultado obtenido y estado (superada/no superada). Puedes usar la plantilla de pruebas que ya tienes; no hace falta crear una nueva.

Se valorará positivamente añadir pruebas automatizadas con JUnit 5, pero para esta entrega reducida no son obligatorias si se documentan y ejecutan correctamente las tres pruebas funcionales.

8. Breve reflexión técnica en README

Añade una sección «UT3: componentes reutilizables» de aproximadamente media página que responda:

- ¿Qué ventaja tiene separar la lectura JSON del menú? ¿Qué inconveniente o coste introduce crear más clases?
- ¿Por qué conviene separar una consulta JDBC de la presentación por consola?
- ¿Qué herramientas has utilizado (IDE, Maven, Javadoc y, si procede, JUnit) y para qué sirve cada una?
- ¿Qué dos opciones de DataStock has modificado para integrar los componentes?

9. Entregables y evaluación

- Repositorio GitHub actualizado, con commits descriptivos y etiqueta de la entrega UT3.
- Proyecto Maven completo: las UT1 y UT2 deben seguir funcionando.
- README actualizado con la breve reflexión y las instrucciones de ejecución que ya existían.
- Registro de las tres pruebas CP-COMP-01 a CP-COMP-03.
- Evidencia de ejecución de las dos opciones integradas (capturas o salida de consola).

En la comprobación individual deberás poder localizar ambas clases, explicar qué responsabilidad tiene cada una y ejecutar las dos opciones del menú. No se exige añadir más funcionalidades que las indicadas.

10. Lista final de comprobación

- ☐ He reutilizado el mismo proyecto DataStock y no he eliminado funcionalidades anteriores.
- ☐ He creado o adaptado un componente JSON sin dependencias de consola.
- ☐ La opción 2 del menú principal utiliza ese componente.
- ☐ He creado o adaptado un componente JDBC que consulta por código con PreparedStatement.
- ☐ La opción 7 > 5 > 2 utiliza el componente JDBC.
- ☐ Los componentes devuelven resultados y los menús se encargan de mostrarlos.
- ☐ He mantenido los mensajes de error rojos y los mensajes correctos verdes.
- ☐ He documentado las clases y los métodos públicos.
- ☐ He realizado y registrado las tres pruebas obligatorias.
- ☐ He completado la reflexión técnica del README.
- ☐ He enviado los cambios a GitHub y comprobado la versión entregada.